COLLEGE OF INFORMATION SCIENCE

CYBR312
Python Scripting in Cybersecurity
Week 4: File Search and Regular Expressions

Cameron Covington • Adjunct Instructor

Week Four Agenda

Week 4

- Processing File Chunks
- Introduction to Hexlify (binascii-standard library)
- File content searching
- Introduction to Python Regular Expressions
- Developing a File Search Script

© 2026 Cameron Covington • Reproduction prohibited • College of Information Science

Week Four Assignments

Week 4

- Your To Do List
- Attend: Week Four Live Lecture
- Watch this week's videos
- Experiment with the sample Scripts
- Quiz 4: File Search
- Assignment 4: Search Files

© 2026 Cameron Covington • Reproduction prohibited • College of Information Science

Videos

Week 4

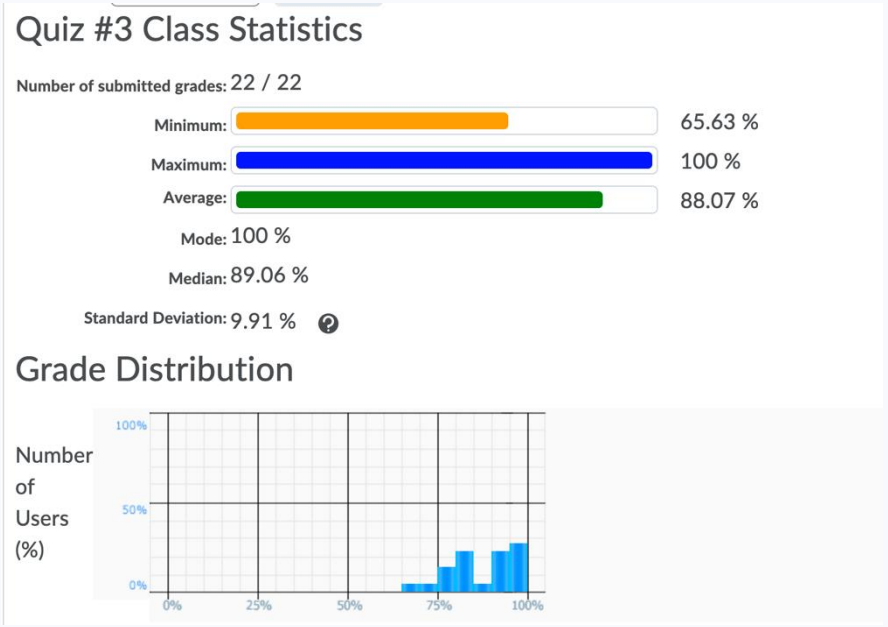
- Week Four Videos
- Quick Search
- Dictionary Introduction

© 2026 Cameron Covington • Reproduction prohibited • College of Information Science

WK-3 Quiz Review

Quiz 3 Review

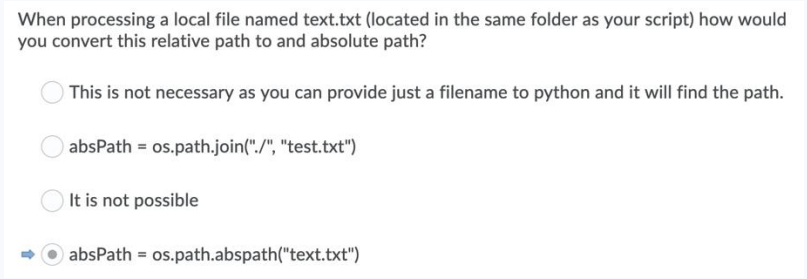
Use this slide as a visual reference during the lecture.

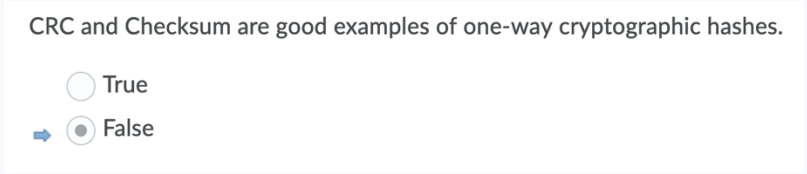


© 2026 Cameron Covington • Reproduction prohibited • College of Information Science

Quiz 3 Review

Use this slide as a visual reference during the lecture.





© 2026 Cameron Covington • Reproduction prohibited • College of Information Science

WK-3 Solution Demonstration

Key Ideas

Week 4

- Topics
- Processing Files in Chunks
- Searching File Contents
- Using Python dictionaries
- What are Regular Expressions
- How to Search a Filesystem

© 2026 Cameron Covington • Reproduction prohibited • College of Information Science

Why read files in chunks?

- Chunking
- Up to this point when reading the contents of files we have attempted to read the entire contents of a file into a variable.
- However, when dealing with large files there may not be enough memory to read the entire file contents all at once. Note on modern system reading 1, 2 or even 128MB of data into a variable works in python, but it is good practice to read in smaller chunks.
- We will add another important twist to this to display the first few bytes of each chunk in hex.

© 2026 Cameron Covington • Reproduction prohibited • College of Information Science

File Chunk Processing Walk-Through

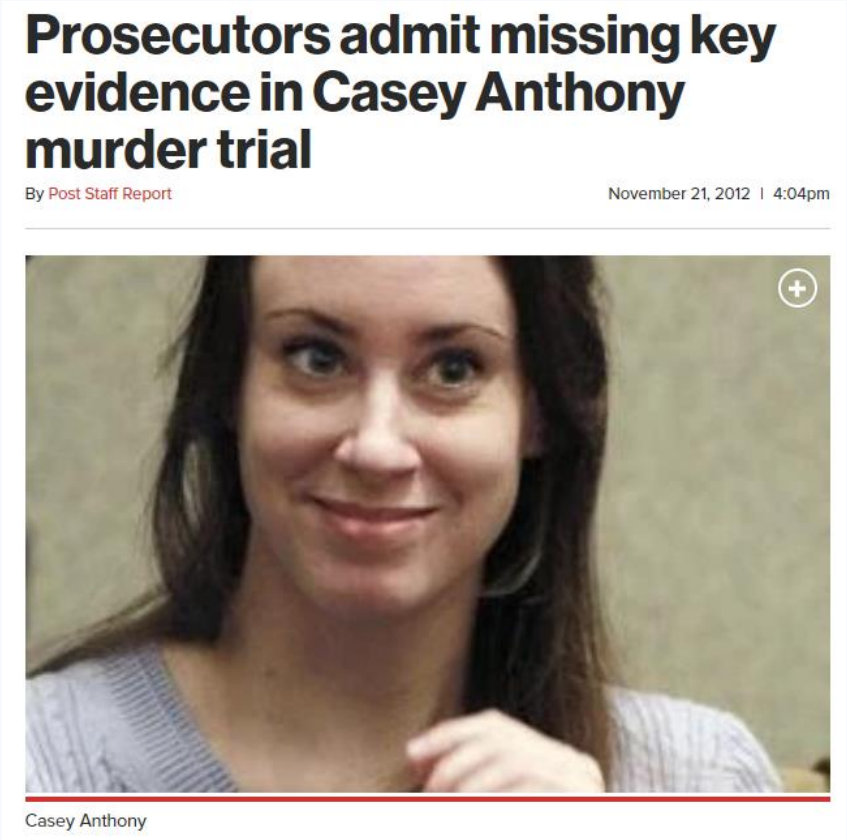
```
'''
Reading Files in Chunks
Professor Hosmer
August 2020
'''
import sys
import os
from binascii import hexlify
print("\nProcessing File Chunks\n")
try:
    # Prompt user for a large file and Chunk Size
    largeFile = input("Enter the name of a large File: ")
    chunkSize = int(input("What size chunks?  "))
    if os.path.isfile(largeFile):  # Verify file is real
        fileSize = os.path.getsize(largeFile)
        # Display details of file
        print("\nProcessing file: ", largeFile)
        print("Total FileSize:    ", "{:,}".format(fileSize))
        print("in chunks of:      ", "{:,}".format(chunkSize),
"Bytes\n")
        bytesProcessed = 0
        # Open and Loop through the file by chunk
        with open(largeFile, 'rb') as targetFile:
```

```
while True:
                fileChunk = targetFile.read(chunkSize)
                bytesProcessed += len(fileChunk)
                if fileChunk:  # if we still have data
                    # Dump the first few bytes of the
chunk
                    hexDump = hexlify(fileChunk[0:16])
                    print(hexDump)
                else:
                    # File has been processed
                    print("\nProcessed:",
"{:,}".format(bytesProcessed))
                    break
    else:
        print(largeFile, "is not a valid file")
        sys.exit("Script Aborted")
except Exception as err:
    sys.exit("\nException: "+str(err)+ "Script Aborted")
print("\nFile Processed ... Script End")
```

© 2026 Cameron Covington • Reproduction prohibited • College of Information Science

Searching Problems

Use this slide as a visual reference during the lecture.



Prosecutors admit missing key evidence in Casey Anthony murder trial

By Post Staff Report November 21, 2012 | 4:04pm

Casey Anthony

© 2026 Cameron Covington • Reproduction prohibited • College of Information Science

Key Issues with Search

- What should we search for?
- How are the search terms structured?
- Could they be misspelled?
- Are they variants?
- What if the suspects use slang?
- What if the format of the files being search is unstructured?



© 2026 Cameron Covington • Reproduction prohibited • College of Information Science

Searching File Contents

- File Searching
- Depending on the file type: i.e. text, photos, executables, documents etc. the approach to searching content is different.
- However, searching for key words or phrases is a good place to start. Note: when processing files like Word documents the contents are compressed; therefore, you would need to decompress the contents first. We will not be covering this advanced topic here.
- The first question is what will we search for? Let's take a simple example to start.

© 2026 Cameron Covington • Reproduction prohibited • College of Information Science

Single Keyword Search

```
'''
Simple Search of a file
Professor Hosmer
August 2020
'''
import sys
import os
print("\nSimple File Search\n")
try:
    # Prompt user for a file and Chunk Size
    largeFile = input("Enter the name of a large File:
")
    chunkSize = int(input("What size chunks?  "))
    keyword   = input("Keyword to Search: ")
    # convert the string to bytes
    keyword = bytes(keyword, 'ascii')
    keyword = keyword.lower() # broaden the search
    if os.path.isfile(largeFile):  # Verify file is real
        with open(largeFile, 'rb') as targetFile:

while True:
                fileChunk = targetFile.read(chunkSize)
                fileChunk = fileChunk.lower()  # broaden
search
```

```
                if fileChunk:  # if we still have data
                    # Search this chunk for our keyword
                    if keyword in fileChunk:
                        print("Found Keyword: ",
keyword)
                        break
                else:
                    # File has been processed
                    print("\nFile Processed:", keyword,
"Not Found")
                    break
    else:
        print(largeFile, "is not a valid file")
        sys.exit("Script Aborted")
except Exception as err:
    sys.exit("\nException: "+str(err)+ "Script Aborted")
print("\nFile Processed ... Script End")
```

© 2026 Cameron Covington • Reproduction prohibited • College of Information Science

Multiple Keyword Search Keywords in a Python list results stored in a Python dictionary

```
'''
Multiple Keyword Search of a file
Professor Hosmer
August 2020
'''
import sys
import os
print("\nSimple File Search v2\n")
try:
    # Prompt user for a file and Chunk Size
    largeFile = input("Enter the name of a large File: ")
    chunkSize = int(input("What size chunks?  "))
    keywordList = [b"jfif", b"test", b"photoshop", b"ishmael"]
    hits = {}  # Create a dictionary to keep track of the hits
    for eachKeyword in keywordList:
        hits[eachKeyword] = 0
    if os.path.isfile(largeFile):  # Verify file is real
        with open(largeFile, 'rb') as targetFile:

while True:
                fileChunk = targetFile.read(chunkSize)
                fileChunk = fileChunk.lower()  # broaden
search
```

```
                if fileChunk:  # if we still have data
                    # Search this chunk for our keyword
                    for eachKeyword in keywordList:
                        if eachKeyword in fileChunk:
                            cnt = hits[eachKeyword]
                            hits[eachKeyword] = cnt + 1
                else:
                    # File has been processed
                    print("\nkeyword \tHits")
                    for key, value in hits.items():
                        print(key,"\t", value)
                    break
    else:
        print(largeFile, "is not a valid file")
        sys.exit("Script Aborted")
except Exception as err:
    sys.exit("\nException: "+str(err)+ "Script Aborted")
print("\nFile Processed ... Script End")
```

© 2026 Cameron Covington • Reproduction prohibited • College of Information Science

Create Possible Word List

```
'''
Create a Possible Word List
Professor Hosmer
August 2020
'''
import sys
import os
import re       # Python regular expression Library
from prettytable import PrettyTable   # 3rd Party Table
Library
                                      # pip install
prettytable
print("\nPossible Word List\n")
try:
    # Prompt user for a file and Chunk Size
    largeFile = input("Enter the name of a large File: ")
    chunkSize = int(input("What size chunks?  "))
    wordDict = {}  # Create an empty dictionary
    if os.path.isfile(largeFile):  # Verify file is real
        with open(largeFile, 'rb') as targetFile:

while True:
                fileChunk = targetFile.read(chunkSize)
                if fileChunk:  # if we still have data
                    txt = re.sub(b"[^A-Za-z\-']", b' ',
fileChunk)
                    txt = txt.lower()  # broaden search
                    wordList = txt.split()
                    for eachWord in wordList:
                        if len(eachWord) >=4 and len(eachWord)
< 15:
                            try:
                                cnt = wordDict[eachWord]
                                cnt += 1
                                wordDict[eachWord] = cnt
                            except:
                                wordDict[eachWord] = 1
                else:
                    # File has been processed
                    tbl = PrettyTable(["Words",
"Occurrences"])
                    for word, cnt in wordDict.items():
                        tbl.add_row([word, cnt])
                    tbl.align = 'l'
                    print(tbl.get_string(sortby="Occurrences",
reversesort=True))
                    break
    else:
        print(largeFile, "is not a valid file")
        sys.exit("Script Aborted")
```

© 2026 Cameron Covington • Reproduction prohibited • College of Information Science

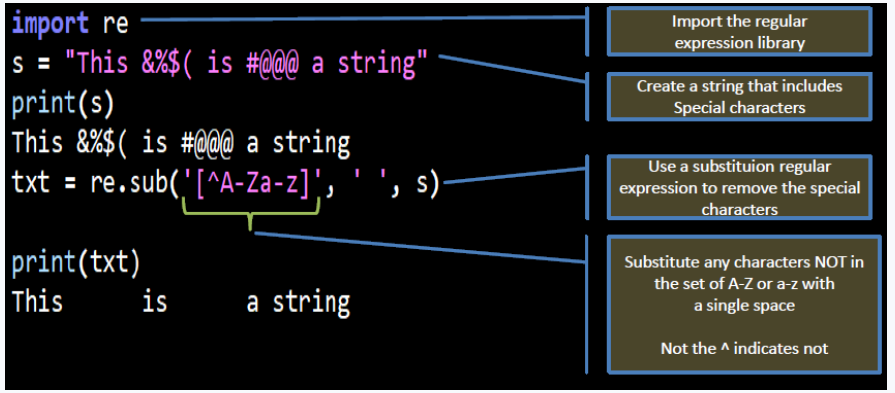
What are Regular Expression How can we use them for search?

- Regular expression or regex for short
- is a sequence of characters that define a search pattern. The patterns are then used by searching methods (in python using the re library) to match specific string formats instead of specific keywords

© 2026 Cameron Covington • Reproduction prohibited • College of Information Science

Simplest Regular Expression Application

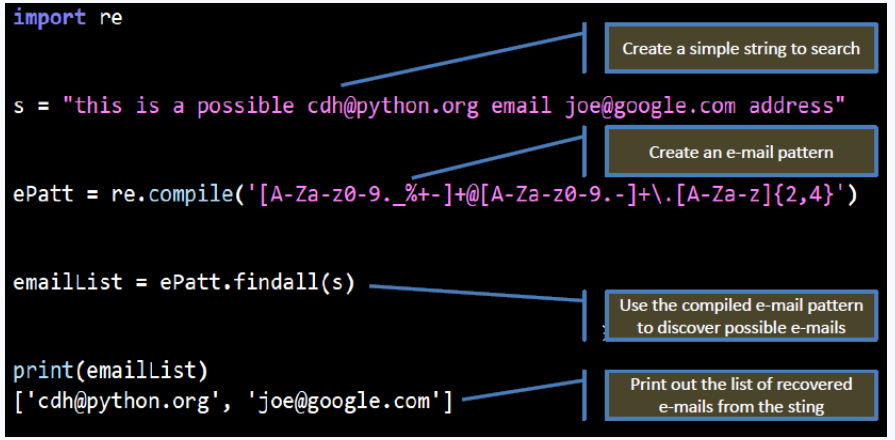
Use this slide as a visual reference during the lecture.



© 2026 Cameron Covington • Reproduction prohibited • College of Information Science

Search for email address

Using Regular expressions



© 2026 Cameron Covington • Reproduction prohibited • College of Information Science

Regular Expressions in Python

```
'''
Regular Expression Search
Professor Hosmer
August 2020
'''
import re
print("\nRegular Expression Example")
emailPattern    = re.compile(b'[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,4}')
urlPattern      = re.compile(b'\w+:\/\/[\w@][\w.:@]+\/?[\w\.?=%&=\-@/$,]*')
try:
    with open("test.bin", 'rb') as targetFile:
        fileContents = targetFile.read()
        emailMatches = emailPattern.findall(fileContents)
        urlMatches   = urlPattern.findall(fileContents)
        print("\nE-MAIL")
        for eachEmail in emailMatches:
            print(eachEmail)
        print("\nURLs")
        for eachURL in urlMatches:
            print(eachURL)
except Exception as err:
    print(err)
```

© 2026 Cameron Covington • Reproduction prohibited • College of Information Science

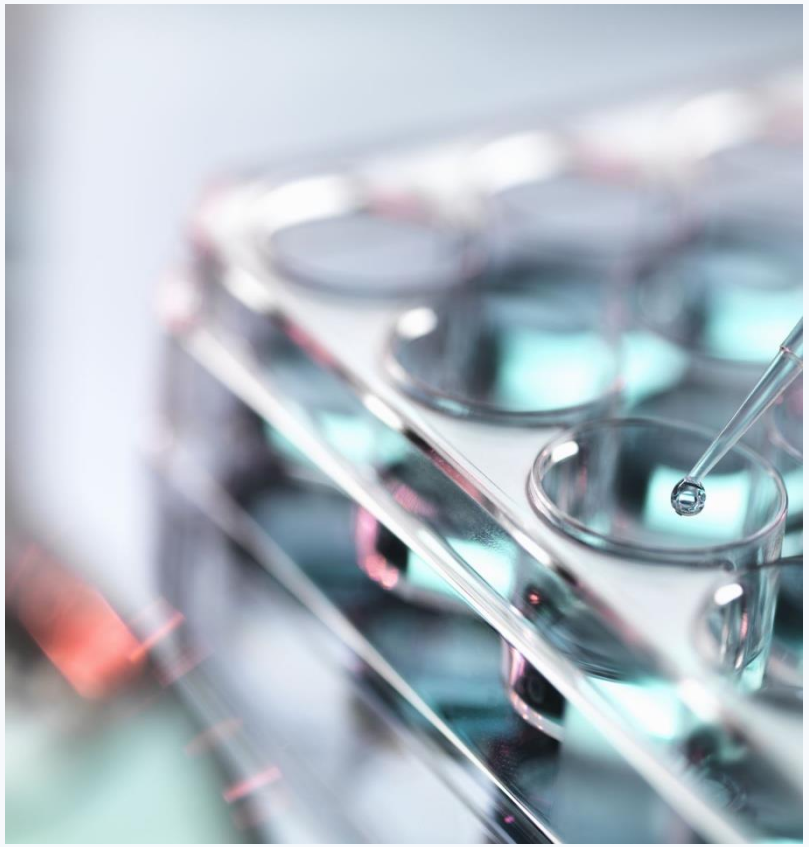
Searching the Filesystem

- Assignment
- Last week you built a script to walk the filesystem starting at specific point and you processed each file and created a prettytable result.
- What if we could expand that script to include search?
- Filename
- File Extension
- File Hash

© 2026 Cameron Covington • Reproduction prohibited • College of Information Science

Live demonstration

Professor Solution



© 2026 Cameron Covington • Reproduction prohibited • College of Information Science

Questions?

Week 4

Coming Next Week

Cracking Passwords with Dynamic Rainbow Tables

© 2026 Cameron Covington • Reproduction prohibited • College of Information Science